



MakersSoc Joystick Control

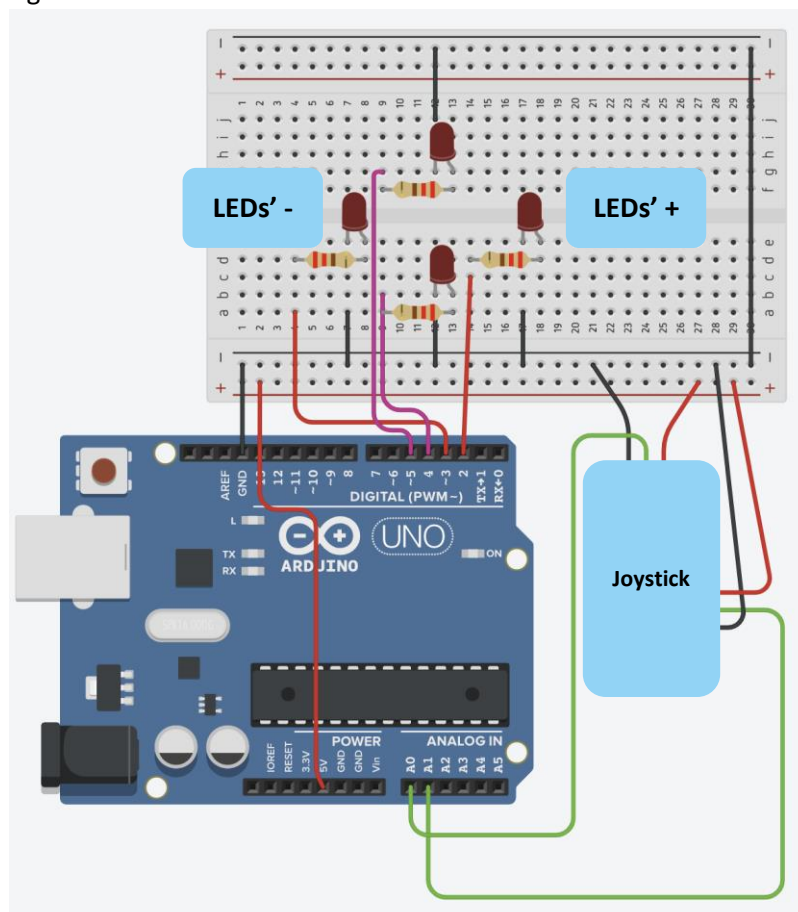
JJ Littleton

The joysticks we will be using are at a basic level, two potentiometers arranged in such a to give axis based inputs. This will be the first steps into processing multiple inputs, though not the most efficient as well as prepare you for different kinds of inputs, as joysticks come on many remote controls at present.

Equipment needed:

Arduino
USB B cable
Arduino IDE
(Personal) Toolkit
Breadboard
Joystick
LED x4
220Ω resistors x4
Wire

1. Assemble this circuit. The LED's polarity is all negative legs are on the left while their positive legs are to the right.





2. Setting up variables

Write the following code:

```
void setup() {  
  Serial.begin(9600); // initialize serial communication at 9600 bits/sec  
  pinMode(2, OUTPUT); // Sets pin as output  
  pinMode(3, OUTPUT); // Sets pin as output  
  pinMode(4, OUTPUT); // Sets pin as output  
  pinMode(5, OUTPUT); // Sets pin as output  
}  
  
void loop() {  
  int sensorValueX = analogRead(A0); // read the input on analog pins  
  int sensorValueY = analogRead(A1);  
  // Convert or 'map' the analog reading (which goes from 0-1023) to integers (2-6)  
  int LEDx = map(sensorValueX, 0, 1023+1, 2, 3+1); //+1 on pins so it is inclusive of  
both pins. +1 on range so it can never activate pin 4 (3+1)  
  int LEDy = map(sensorValueY, 0, 1023+1, 4, 5+1); //Sets it to the correct pin  
  // print out the LED value you read:  
  Serial.println(sensorValueX);  
  Serial.println(sensorValueY);  
  Serial.println(LEDx);  
  Serial.println(LEDy);  
}
```

Setting up the I/O and printing to the serial monitor.

The values can be read in the serial monitor accessible through going to the "Tools" menu and select "Serial Monitor". Make sure the board is connected.

The values from the 0-1023 range have their equivalent found within the range representing the output pins, "2, 3+1" & "4, 5+1".



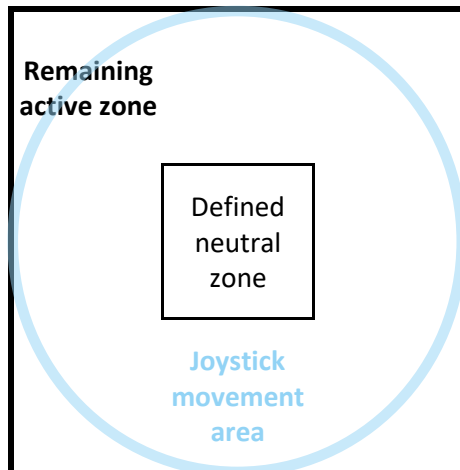
3. Neutral Zone

Next, write the following code at the end of the loop function:

```
//Declare neutral zone config in scope
int neutralZoneDiameter = 200; //Equal on both axis. Uses same on both axis
int neutralZoneRadius = round( (neutralZoneDiameter)/2 );

//Set up variables to reduce repetition & increase readability
int centre = round( (1023-0)/2 ); //for both axis
int neutralLower = centre - neutralZoneRadius;
int neutralUpper = centre + neutralZoneRadius;
```

This allows for the joystick to have a state where it does not active any components. Later, the sensed value will be checked to see if it is within a certain range. It will essentially simulate as shown in the diagram below.





4. Lighting LEDs

Next, write the following code continuing from the end of and within the loop function:

```
//Light up LEDs
if (neutralLower < sensorValueX && sensorValueX < neutralUpper) { //In neutral zone
  digitalWrite(LEDx, LOW);
} else { //Not in neutral zone
  digitalWrite(LEDx, HIGH); // Turn the relevant LED on
}

if (neutralLower < sensorValueY && sensorValueY < neutralUpper) { //In neutral zone
  digitalWrite(LEDy, LOW);
} else { //Not in neutral zone
  digitalWrite(LEDy, HIGH); // Turn the relevant LED on
}

delay(10); // Wait 100 ms
digitalWrite(LEDx, LOW); // Turn the LED off
digitalWrite(LEDy, LOW);
```

The if statements are essentially “if within range” conditions with the sensed value being checked against the lower bound and upper bound. These are then within an AND gate, only allowing the condition to be true if both conditions are met. If the value is both greater than the lower bound AND lower than the upper bound.

This code should light up the LEDs depending on the direction you point the joystick. If the LED does not align/match with the direction the joystick is in, there are a number of ways to fix or address this.

- Rewiring – the wires connecting the components can be rearranged.
Swapping the A0 and A1 joystick wires would swap the axis.
Similarly, rewiring the LEDs’ connection to the Arduino will change their output.
- Code – changing the inputs read and outputs written to within the code could solve problems.

**5. Your final code should look like this:**

```
void setup() {
  Serial.begin(9600); // initialize serial communication at 9600 bits/sec
  pinMode(2, OUTPUT); // Sets pin as output
  pinMode(3, OUTPUT); // Sets pin as output
  pinMode(4, OUTPUT); // Sets pin as output
  pinMode(5, OUTPUT); // Sets pin as output
  int LEDx; // Define an integer variable "LEDx"
  int LEDy; // These are used to control LED per axis is lit
}

void loop() {
  int sensorValueX = analogRead(A0); // read the input on analog pins
  int sensorValueY = analogRead(A1);
  // Convert or 'map' the analog reading (which goes from 0-1023) to integers (2-6)
  int LEDx = map(sensorValueX, 0, 1023+1, 2, 3+1); //+1 on pins so it is inclusive of both
pins. +1 on range so it can never activate pin 4 (3+1)
  int LEDy = map(sensorValueY, 0, 1023+1, 4, 5+1); //Sets it to the correct pin
  // print out the LED value you read:
  Serial.println(sensorValueX); //debugging
  //Serial.println(sensorValueY); //debugging
  Serial.println(LEDx);
  //Serial.println(LEDy);

  //Declare neutral zone config in scope
  int neutralZoneDiameter = 200; //Equal on both axis. Uses same on both axis
  int neutralZoneRadius = round( (neutralZoneDiameter)/2 );

  //Set up variables to reduce repetition & increase readability
  int centre = round( (1023-0)/2 ); //for both axis
  int neutralLower = centre - neutralZoneRadius;
  int neutralUpper = centre + neutralZoneRadius;

  //Light up LEDs
  if (neutralLower < sensorValueX && sensorValueX < neutralUpper) { //In neutral zone
    digitalWrite(LEDx, LOW);
  } else { //Not in neutral zone
    digitalWrite(LEDx, HIGH); // Turn the relevant LED on
  }

  if (neutralLower < sensorValueY && sensorValueY < neutralUpper) { //In neutral zone
    digitalWrite(LEDy, LOW);
  } else { //Not in neutral zone
    digitalWrite(LEDy, HIGH); // Turn the relevant LED on
  }

  delay(10); // Wait 100 ms
  digitalWrite(LEDx, LOW); // Turn the LED off
  digitalWrite(LEDy, LOW);
}
```